Folio

A course assistant inside IntelliJ.
It answers from the lectures, and it shows the page.

罗臻 Luo Zhen
2451989

李灿文 Li Canwen
2452270

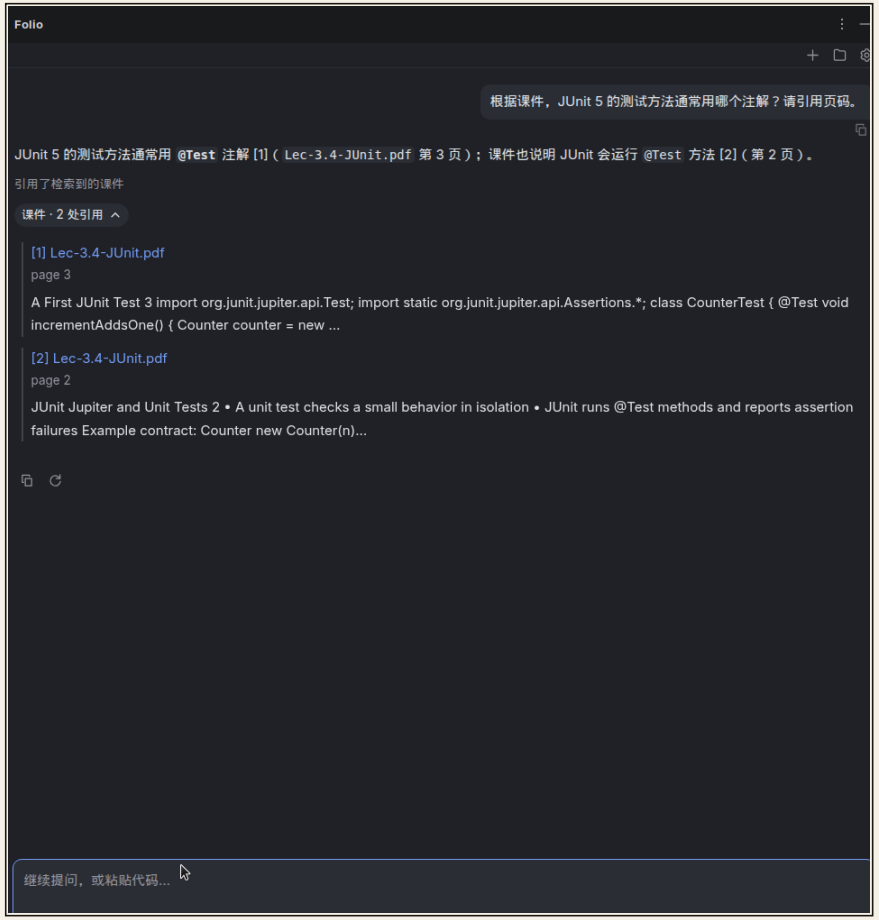
范羽 Fan Yu
2452750

孟圣雨 Meng Shengyu
2452272

What it is

A tool window on the right of IntelliJ.
It answers from the lectures on this machine,
and it names the page.

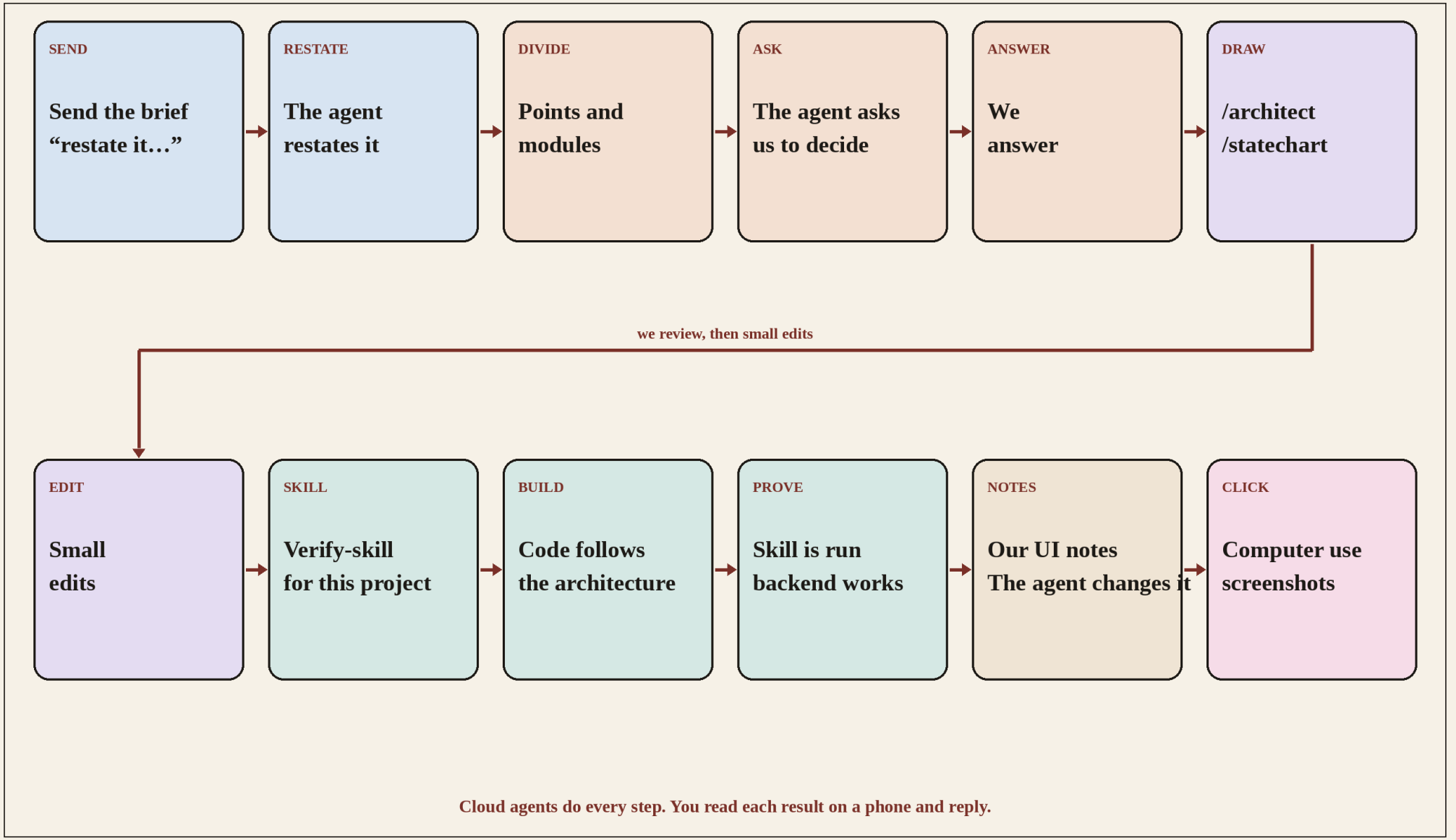
The picture is that answer: @Test,
from Lec-3.4, pages 3 and 2.



F0	Course retrieval. Chunk, index, retrieve, cite the page.
F1	A drill. A JUnit preview written only when asked.
	Commit text from the staged diff.
Not ours	Group C rewrites code and drives the diff viewer.

How it was made

03 MAKING



Decisions that stayed with us

After the restatement, the agent named the choices it would not make alone.

Retrieval	A local word index, not an embedding service.
The third action	A course drill. Not Group C, and not another ask.
Tests	Show a preview. Write the file only after a click.
Commits	Show the message. Do not run git commit.
The model	The address stays in settings. This demo uses DeepSeek.

The library, and a question

04 THE WINDOW

You bring the lectures in from the course-files button.

Folio accepts PDF, PPTX, TXT, Markdown, and DOCX.

It reads them on this machine. Indexing needs no key.

The library is shared by every project in this IDE.

Removing a file asks first. The original stays on disk.

You ask in the box, or about selected code.

The answer arrives as it is written.

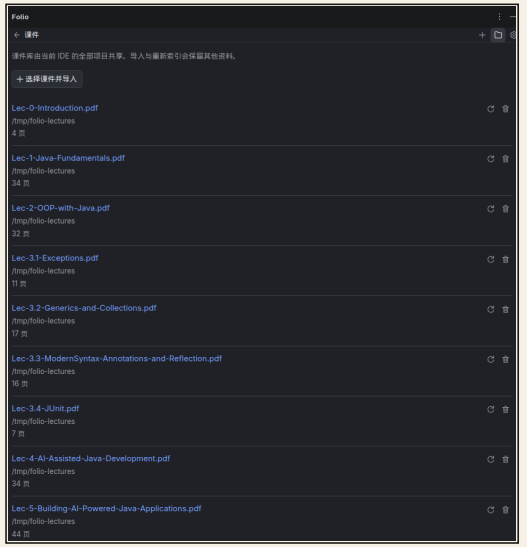
Thinking stays in its own block.

If you stop, the words already shown remain.

A used page is marked [1]. The title opens it.

A miss is still an ordinary answer. No invented page.

With no key, search runs and no model call is made.





Practice, settings, and drafts

04 THE WINDOW

Practice gives three exercises, from a topic or from selected code.

They appear together only after the reply parses as three exercises.

Until then you see a status line, not a stream of tokens.

You open one explanation at a time. The editor is not changed.

Settings holds the address and the model.

The key is in the operating system keyring.

After it is saved, the field is empty.

The line under it says a key for this address is saved.

Indexing never needs that key. Asking the model does.

Ask, practice, tests, and commit text each keep a draft.

Changing mode does not throw a draft away.

A new chat clears the transcript. The drafts stay.

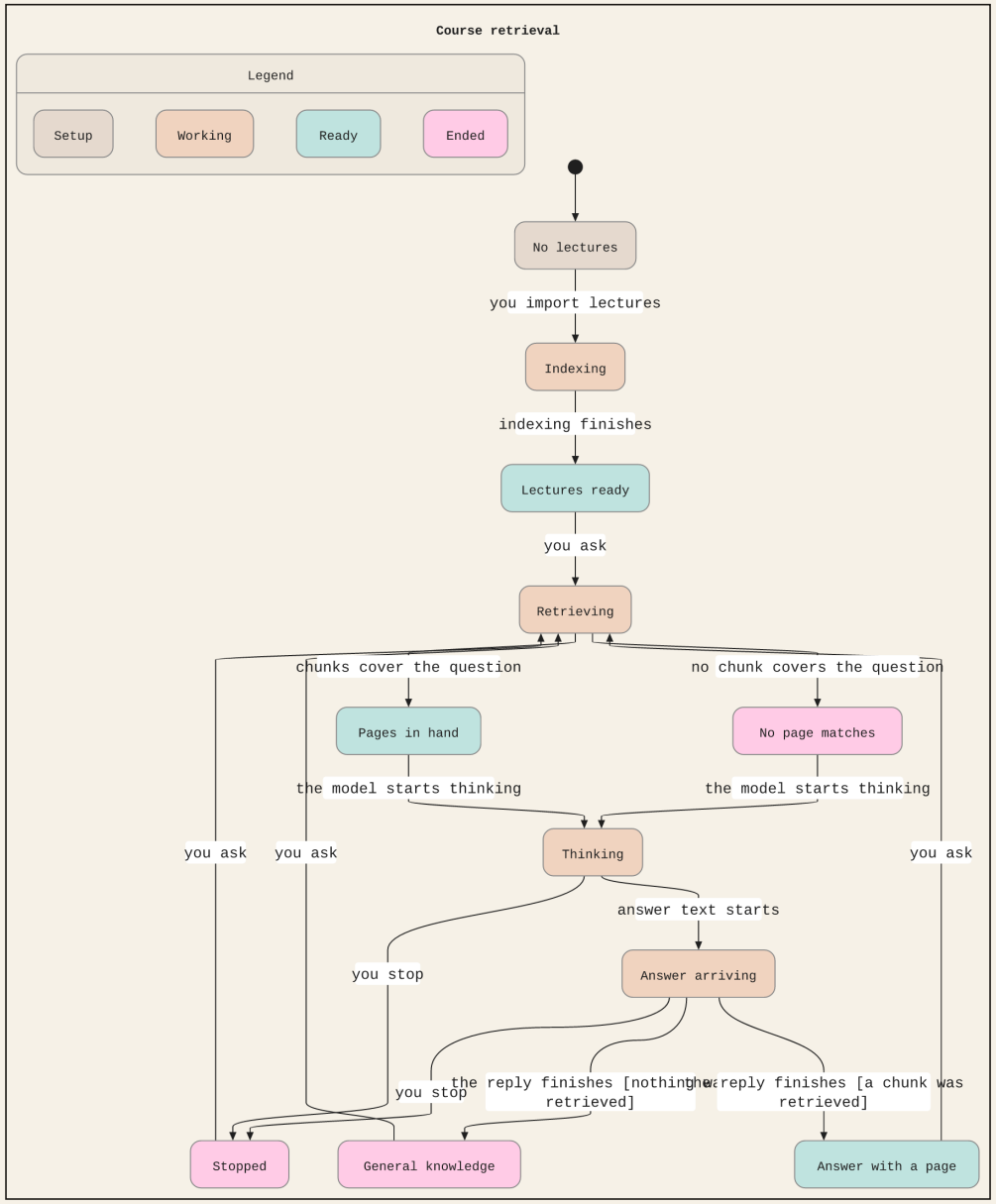




How a file becomes a page

05 HOW IT WORKS

Folio does not invent page numbers.
A PDF keeps its own pages, including blanks.
A slide deck keeps slide order.
A text file is a section, not a made-up page.
Each of those is cut at about 700 characters.
The next piece starts 100 characters earlier,
so a sentence on the cut is not lost.
The pieces are indexed here, by their words.
There is no embedding model and no web search.
A question, plus any selected code, is matched.
Folio keeps at most four pieces.
A piece must cover at least half the terms.
One shared word in a longer question misses.
The model writes [1] for a piece it uses.
[1] becomes that file and page. The title opens it.
Stopping does not invent a page.
The drawing is the same rule.



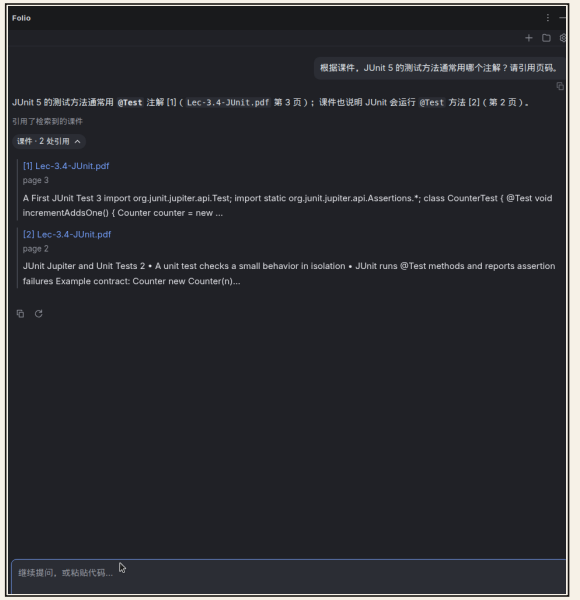
A Chinese question, English lectures

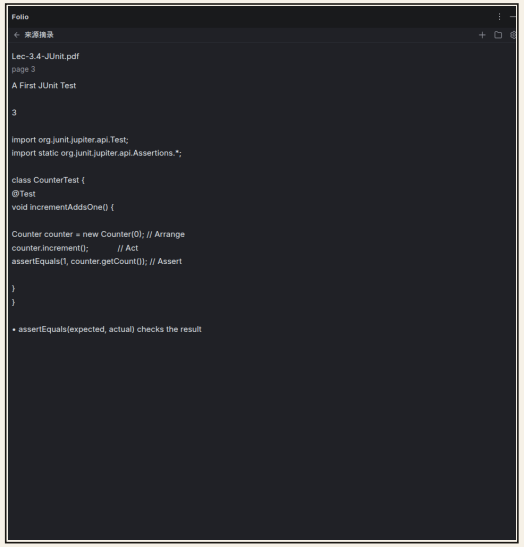
05 HOW IT WORKS

The question was:

根据课件，JUnit 5 的测试方法通常用哪个注解？

The lectures are in English.
The Chinese parts do not occur on any page.
A piece must cover at least half the question.
If the Chinese also had to be covered,
the search would return nothing.
Folio sets that Chinese aside only when both
of these are true: none of it is in the library,
and a Latin word, here JUnit, is in the lectures.
The search is then judged on JUnit.
The reply cites Lec-3.4, pages 3 and 2.
No Latin word from the lectures: this path
is not used. One Latin word inside a long
unrelated question is still not enough.





Two ways the project can change

05 HOW IT WORKS

Tests

You describe what should be tested.
Folio shows a Java preview.
It checks syntax and JUnit shape.
It does not compile and it does not run.
The file appears only when you click Write.
You can undo that write.
Cancel, or a preview that is not a test,
writes nothing.
The chart: write is reachable only
from a preview that parsed as Java.

Commit text

Staging is a button you press.
Folio reads that staged diff.
It shows three candidate messages.
You copy one, or fill the commit box.
If the staged files have changed,
copy and fill are refused.
Drafting never runs git commit.
The chart has no step that commits.

Asking and practice never write a file and never produce a commit.

